

Extrinsic Safeguarding Mechanism Requirements for Trustworthy Physical AI

Matthew Harris
SAIF Autonomy
London, United Kingdom
matt@saifautonomy.ai

Dewi Madden
SAIF Autonomy
London, United Kingdom
dewi@saifautonomy.ai

Abstract— As embodied AI systems increasingly control real-world actuation, across domains from autonomous vehicles to industrial robotics and uncrewed aircraft, the need for system-level safeguards that go beyond model-centric assurance is needed. This short paper argues that *extrinsic safeguarding mechanisms*, such as runtime assurance (RTA) architectures, are essential to enabling trustworthy and certifiable physical AI. While standards like ASTM F3269-21 formalise architectural elements and requirements for runtime assurance in aviation, similar principles are emerging across automotive (UL 4600, ISO 21448), industrial robotics (ISO 10218), and functional safety (IEC 61508, ISO 13849). Yet, a gap persists: current frameworks lack a unifying set of higher-level system requirements for extrinsic safeguarding mechanisms to inform OEM and operator implementation across diverse domains. Drawing on recent literature from NIST, NASA, and academic work on Simplex and safety cage architectures, we synthesise a cross-domain perspective on how layered, independent safeguarding systems can act as “*trust anchors*”, bounding AI behaviour during runtime and supporting continuous verification against system-level safety constraints. We propose a set of cross-cutting requirements for such safeguards, including monitor-controller independence, verifiable fallback paths, and runtime enforcement of safety envelopes. This position aligns with emerging guidance from ISO/IEC TR 5469:2024 and IEEE P7009, and builds on the practical RTA implementations developed by SAIF Autonomy. We advocate for the standardisation of extrinsic safeguard requirements as a foundation for building justified trust in Physical AI.

Keywords— *Runtime Assurance, Autonomous Systems, AI Assurance, Safeguards, Guardrails, UAV*

I. INTRODUCTION

Physical AI systems, such as autonomous vehicles, drones, and intelligent robots, hold great promise but also pose significant safety risks due to their complexity and learning-driven behaviours. Recent incidents (e.g., an autonomous vehicle fatality in Arizona in 2018 and the suspension of a robotaxi service in 2023 after pedestrian injuries[1]) have underscored the potential for catastrophic failures when AI operates in the physical world. These events erode public trust and highlight that traditional design-time safety assurances alone are insufficient. In response, there is growing consensus that extrinsic safeguards are needed to contain AI behaviours within safe bounds for AI-based systems[2]. Even high-level AI

ethics guidelines emphasise resilience and a “fall back plan” for safety-critical AI systems [3].

Extrinsic safeguarding mechanisms refer to engineered risk controls that can be applied to AI-based systems [2], such as runtime assurance (RTA) mechanisms; independent monitors and safety controllers that supervise the primary AI controller. Unlike intrinsic safety (ensuring the AI model is safe-by-design), extrinsic safeguards provide an external safety net: they continuously check the AI’s actions against safety constraints and intervene (override or adjust control) if an unsafe condition is detected. Fig. 1 illustrates the difference between extrinsic vs intrinsic safeguards. This approach draws on proven safety paradigms (e.g., the Simplex architecture that pairs a complex controller with a high-assurance fallback controller[4]). By deploying an independent safety monitor alongside the AI, we can contain unpredictable behaviours and achieve fail-safe or fail-operational outcomes even if the AI misbehaves. Such runtime mechanisms are crucial for stakeholder trust, as they assure regulators and users that even unforeseeable AI faults will be caught by a certifiable safety backstop.

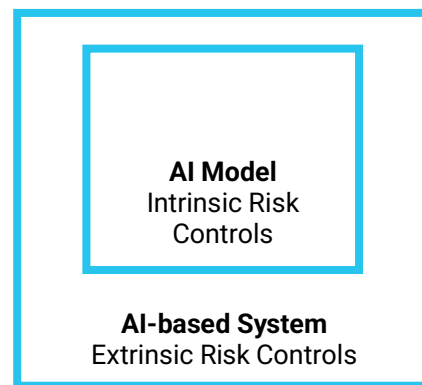


Fig. 1 Intrinsic vs Extrinsic Risk Controls [2]

This paper argues for the need of extrinsic safeguarding mechanisms in AI-enabled physical systems and proposes a set of requirements to guide their development and certification. We survey related standards and work in multiple domains (aerospace, automotive, robotics, etc.), identifying gaps in

current safety requirements. We then introduce formal requirements for certifiable RTA safeguards, including monitor-controller independence, reliable fallback strategies, and runtime enforceability of safety limits, with the goal of meeting the highest levels of certification (e.g., DAL A under DO-178C). We discuss how these requirements support a safety assurance framework that can foster stakeholder trust and enable the safe deployment of Physical AI. Figures are included to illustrate the RTA architecture and cross-domain safeguarding concepts. The overall contribution is a cross-industry synthesis of best practices into concrete requirements for extrinsic safeguards to achieve trustworthy physical AI.

II. BACKGROUND AND RELATED WORK

A. Safety Standards and Guidelines

A number of standards have emerged to address safety of autonomous and robotic systems, but each has a specific focus and leaves certain gaps at the full system level. In aviation, ASTM F3269-21 “Standard Practice for Methods to Safely Bound Flight Behavior of Unmanned Aircraft Systems Containing Complex Functions Using Run-Time Assurance” was introduced as guidance for integrating a runtime safety monitor in unmanned aircraft[4]. This standard essentially proposes a certification strategy wherein a complex function (e.g. an AI or machine learning-based controller) is supervised by a high-assurance safety monitor that bounds its behaviour [5]. The safety monitor can intervene or override to keep the system within predefined safety limits, an approach akin to the Simplex architecture. ASTM F3269-21 thus provides a framework for design-time approval of advanced functions by runtime containment, however, it is focused on aerial vehicles and does not enumerate detailed cross-domain requirements for the monitor itself (e.g., software integrity or independence requirements beyond the aviation context).

In the automotive domain, the UL 4600 standard (ANSI/UL 4600: Standard for Safety for the Evaluation of Autonomous Products) takes a safety case approach for autonomous vehicles. Rather than prescribing specific mechanisms, UL 4600 prompts manufacturers to ensure all potential assurance gaps are addressed[6]. This includes considering fail-safe architectures and fallback behaviours if the driving AI fails. For example, UL 4600 expects that the safety case will answer how the system handles situations beyond the AI’s capabilities or when it malfunctions [7]. However, UL 4600 stops short of defining concrete technical requirements for a runtime monitoring subsystem as it is largely process-oriented, requiring that if runtime risk mitigation is needed, it should be part of the safety case. ISO 21448:2022 (Safety of the Intended Functionality – SOTIF) [8] complements the functional safety standard (ISO 26262) [9] by focusing on unknown unsafe scenarios, and it explicitly recommends operation-phase monitoring to detect and address new hazardous conditions that were not identified prior to deployment. SOTIF thus acknowledges that even after design-time validation, unforeseen situations can arise in the field, runtime monitoring mechanisms are needed to catch these and initiate appropriate safe responses. Again, the standard highlights what should be done (identify and mitigate runtime hazards) but not how to implement an independent safeguard system.

Industrial automation and robotics safety standards have long mandated extrinsic safety measures, though in a more hardware-centric fashion. ISO 10218-1 (Safety Requirements for Industrial Robots) [10] requires that safety-related parts of the control system meet high integrity levels (e.g., Performance Level d, Category 3 per ISO 13849 [11], or SIL 2 with single-fault tolerance per IEC 61508 [12]). In practice, this means an industrial robot must have an independent, reliable safety controller (or circuit) that can, for instance, stop the robot’s motion if a limit is exceeded or if a person enters a hazardous area. Traditional measures include physical guards, emergency stop circuits, and safety PLCs, effectively external mechanisms ensuring the robot cannot exceed certain speeds or forces when humans are present. While these measures handle many foreseeable hazards, they were not originally conceived for AI-driven robots that dynamically plan actions. As robots become more autonomous and collaborative, there is a need to extend these standards with runtime AI supervision, for example, a software monitor that watches the robot’s decisions and intervenes if an unsafe manoeuvre is about to occur. Existing robot standards provide the baseline (hard limits, redundant safety channels) but lack explicit guidance on AI decision monitoring or the interplay between a learning controller and the safety system.

Functional safety frameworks like IEC 61508 (general E/E/PE safety) and its sector-specific derivatives (e.g., ISO 26262 for automotive) generally advocate fail-safe design and independence of safety functions. The concept of a separate safety layer that can take corrective action on hazardous failures is inherent in these standards. However, applying these to AI controllers raises challenges – a machine-learned controller might not have a clear failure mode or may behave unpredictably in novel conditions. Thus, ensuring an independent safety function that covers all AI failure modes is non-trivial. Recent research and guidance have started to fill this gap. For instance, NASA has published preliminary guidance on runtime assurance for autonomous aerospace systems[13], recommending requirements for the runtime monitor (e.g. independence from the primary system, assured input data, complete coverage of the operational state space, and predictable performance vs. false-alarm trade-offs). Academic works on runtime verification and the Simplex architecture have demonstrated that a well-designed external safety envelope can keep systems safe even when using complex, unverified controllers [14]. The literature also stresses that the safety monitor itself becomes a single-point-of-failure if not engineered to high standards; the monitor must be simple enough to verify, because the overall system safety hinges on its correctness [13].

B. Gaps and Needs

In summary, existing standards across domains recognise aspects of runtime safety (e.g., requiring consideration of failsafes or stating high-level need for monitoring), but none provide a unified, full-spectrum requirement set for an extrinsic AI safeguard. There is a clear gap in system-level requirements: How exactly should the monitor and fallback be designed, what properties must they have to be certifiable, and how to ensure they indeed cover the “unknown unknowns” introduced by AI? The related work indicates that a runtime safeguard should be

independent, reliable, and able to handle the worst-case hazards, but developers lack detailed guidance on implementing such mechanisms in a certifiable way. This paper addresses that gap by drawing together cross-domain insights and proposing concrete requirements for extrinsic safeguarding mechanisms.

III. THE CASE FOR EXTRINSIC SAFEGUARDS

AI-enabled systems operating in the open world face uncertainties and novel situations that cannot be completely anticipated during development. Intrinsic safety techniques – such as extensive training, verification, or simulation testing of the AI model – help mitigate risk but do not provide enough confidence in the safety of the system, especially for safety-critical applications. There will always be conditions under which even a well-designed AI could make an unsafe decision (e.g., misclassifying an obstacle, or choosing an unstable control action due to a rare combination of sensor inputs). Extrinsic safeguards serve as a last line of defence to handle such situations at runtime [2]. They bring several key benefits and rationale:

A. Bounding Unpredictable Behaviour

A runtime monitor can enforce fundamental safety constraints (like “never exceed safe speed” or “maintain safe separation distance”). Even if the AI’s internal logic is complex or non-transparent, the monitor observes its outputs and the system state to ensure these outputs won’t lead to a violation of those constraints. For example, if an autonomous drone’s learning-based controller issues a command that would send it outside a geo-fenced area, an external geofence monitor can catch this and command a safe action or return-to-base [4]. By safely bounding the AI’s behaviour, we can allow advanced AI functions to operate within a “guarded” envelope, gaining performance benefits of AI while containing risk [15].

B. Fail-Safe and Fail-Operational Response

Extrinsic safeguards typically implement fallback strategies when an unsafe condition arises. This could be a simple fail-safe (e.g., emergency stop or shut-down in a robot), or a fail-operational transition (e.g., switching to a safe backup controller that can keep the system running in a degraded but safe state). Having an automatic fallback mitigates the risk of catastrophic failure. Notably, in aviation and automotive history, human operators have often served as the fallback (pilots overriding autopilot, drivers taking control from ADAS). In fully autonomous systems, an RTA mechanism assumes that role. It detects emergencies faster than a remote human could and acts decisively to prevent accidents. This immediate containment is critical for trust: stakeholders need to know that if the AI goes wrong, there is a mechanism that will promptly stabilise the situation or bring the system to a safe halt.

C. Assurance for Certification and Trust

From a certification and regulatory perspective, extrinsic safeguards provide a compelling assurance argument. Regulators like aviation authorities are more willing to certify novel AI-based functions if an independent safety layer guarantees that any AI malfunctions cannot lead to an unsafe state [13]. The runtime monitor essentially isolates the high risk from the AI: even if the AI is not provably correct, the overall system can be provably safe because the monitor/fallback will

override dangerous commands. This idea is explicitly the basis of ASTM F3269-21’s certification strategy using RTA. In the automotive sector, a similar logic underpins safety cases – a manufacturer can argue that an autonomous vehicle is safe because it has, say, an independent collision avoidance module that will always prevent certain types of crashes, even if the main driving policy fails. Moreover, demonstrating such concrete protective measures greatly improves stakeholder confidence. Passengers, developers, and insurers can trust the system more knowing a “guardian” is always watching the AI’s actions. In essence, extrinsic safeguards translate into a form of operational guarantee that complements statistical or design-time assurances: no matter how the AI learns or evolves, these hard constraints will not be violated.

D. Handling Unknown Hazards and SOTIF issues

One of the strongest arguments for runtime safeguards is their ability to handle the “unknown unknowns.” Design-time safety analysis (like FMEA or simulation) might miss certain rare scenarios. ISO 21448 (SOTIF) explicitly calls for monitoring during operation to catch new unsafe scenarios [8]. For instance, suppose an autonomous car encounters a new type of object or an unusual weather phenomenon that confuses its vision system – an intrinsic design might not have accounted for it. But an extrinsic monitor could detect anomalies (like the car failing to detect an obstacle or the vehicle dynamics becoming erratic) and then trigger a risk mitigation (e.g., slow down or pull over). In this way, the system can adapt and remain safe even when confronting surprises. The runtime safeguard essentially provides a safety net for the unknown, buying time for engineers to update the AI after such incidents are observed.

E. Summary

In summary, extrinsic safeguards are essential for bridging the gap between AI’s unpredictable competence and the high assurance required in safety-critical physical systems. They implement the *defense-in-depth* principle: even if the primary AI control layer has flaws, a secondary independent layer will catch and correct those flaws before harm occurs. By doing so, they not only prevent accidents but also enable innovation – developers can deploy powerful AI controllers sooner, without waiting for impossible-to-achieve perfect assurance, as long as a certifiable safety wrapper contains the risk. The next sections examine how different domains realise these principles and then propose a unified set of requirements to design such safeguard mechanisms in a certifiable manner.

IV. CROSS-DOMAIN REVIEW OF REQUIREMENTS

Extrinsic safety mechanisms manifest in various forms across different industries. We review how key requirements for runtime safeguards are addressed in aviation, automotive, and industrial domains, extracting common principles and gaps that inform our proposed requirements.

A. Aerospace

The concept of a safety monitor overriding a primary controller has been explored extensively in aerospace, where failure can be catastrophic. Aside from ASTM F3269-21’s formalisation of RTA for unmanned aircraft, earlier implementations existed in specific systems. For example, the Automatic Ground Collision Avoidance System (Auto-GCAS)

in fighter jets monitors the aircraft's trajectory and will autonomously execute a recovery pull-up if it detects an imminent ground collision, even if the pilot (or autopilot) fails to act [4]. This is an extrinsic safety function with a strict requirement: it must reliably detect the hazard (upcoming ground impact) and intervene in time, without itself triggering unnecessarily in normal flight. The requirement of timely hazard detection and intervention is evident – the monitor must operate in real-time with a worst-case response time bounded such that it always can prevent the accident. Another requirement is independence and authority: Auto-GCAS is implemented on a separate logic from the flight control system and has direct authority to override the controls when activated. Modern RTA frameworks in aerospace similarly require that the safety monitor be sufficiently independent of the complex function it oversees, to avoid common-cause failures, and that it has the authority (through a dedicated actuator or command channel) to enforce safe outcomes (e.g., engaging a safe mode or backup controller). Aviation standards like DO-178C [16] further demand that any safety-critical software (such as the monitor) meets rigorous development assurance, for a monitor that guards against catastrophic failures, a Design Assurance Level A (DAL A) is typically needed. DAL A software must demonstrate extremely high reliability (failure probability $\leq 10^{-9}$ per hour) through verification and validation [17]. In practice, this means the monitor's requirements, design, code, and tests are all subject to intense scrutiny, including traceability and coverage of possible cases, to ensure it will not fail to act when needed (or act erroneously when not needed). To summarise the aerospace perspective: (a) monitors should be independent, high-assurance components (DAL A equivalent) that supervise the AI; (b) they must enforce critical safety constraints (flight envelope limits, geofencing, collision avoidance) with direct override capability; (c) they should be proven to cover all relevant hazard scenarios identified in the safety analysis (and ideally be designed conservatively to cover even unmodeled scenarios within reason).

B. Automotive

In current advanced driver-assistance systems (ADAS) and autonomous driving prototypes, we also see emerging forms of extrinsic safeguards, though the domain historically relied on the human driver as the ultimate fallback. One example is automatic emergency braking (AEB) systems: these act as an independent collision mitigation layer that will apply brakes if an imminent crash is detected, even if the human driver (or an autopilot feature like adaptive cruise control) fails to brake in time. AEB can be seen as a simple runtime monitor focused on forward collision safety – it monitors sensors (radar, camera) for obstacle distance and intervenes on the braking actuator if thresholds are violated. Key requirements illustrated by AEB include monitoring a critical parameter continuously (headway distance and closing speed), and taking control authority when a safety threshold is crossed (deceleration commanded to avoid collision). As vehicles move toward higher autonomy (SAE Level 4/5 with no human driver fallback), more comprehensive runtime assurance will be needed. For instance, an autonomous shuttle might have a supervisory system that ensures it stays within a geo-fenced area and obeys speed limits, akin to how aviation RTAs enforce geofences and speed constraints.

Automotive safety standards are evolving: ISO 26262 (functional safety) provides the need for safety mechanisms and ASILs (Automotive Safety Integrity Levels) but focuses on hardware/software faults rather than AI decision errors; ISO 21448 (SOTIF) explicitly calls for monitoring performance limitations[8]. A concrete requirement emerging in automotive discussions is fail-operational capability, if an autonomous driving function fails or is aborted by a monitor, the vehicle should execute a minimal risk manoeuvre (like safely pulling over) rather than simply disengaging and causing chaos. Thus, the extrinsic safeguard in an AV likely needs a fallback motion planner/controller that can take over and achieve a safe stop or safe state. Another requirement is sensor and compute redundancy for the monitor: the monitor might use a separate set of simpler sensors or algorithms to cross-check the AI's perceptions and decisions. This reduces the chance that a single fault (e.g., a common sensor failure or common algorithm flaw) affects both the AI and the monitor. For example, if the main AI relies heavily on camera vision, a monitor might rely on a simpler lidar or radar-based logic to detect imminent collisions, providing diversity. In summary, automotive domain reinforces the need for (a) effective hazard detection (like collision detection) with possibly diverse sources, (b) independent actuation to enforce safety (braking, steering to avoid accidents), (c) a well-defined safe-state transition (minimal risk condition) as the fallback action, and (d) high reliability of the safeguard, potentially up to SIL3/ASIL D levels for life-threatening hazards, which parallels DAL A in rigor.

C. Industrial Robotics and Machinery

As noted earlier, industrial safety standards mandate independent safety circuits. A classical requirement in machinery safety (per IEC 61508/ISO 13849) is that any safety function (emergency stop, overspeed limiter, etc.) must be designed to meet a certain Safety Integrity Level with independence from the normal control system [12][11]. In practice, an industrial robot cell might have a safety PLC that constantly monitors the robot's joint speeds and positions and will cut power if limits are exceeded or if a person steps past a light curtain sensor. The integrity and response time requirements here are strict: for instance, stopping a robot before it can injure a person might require the safety system to react within a few milliseconds once a human is detected in the danger zone. Additionally, fail-safe design is a cornerstone, e.g., safety relays are normally closed and power is removed (causing stop) if any fault is detected (loss of power, broken wire, etc.). This principle can inform AI safeguards: the monitor and its actuators should be designed such that any failure tends toward a safe state (for example, if the monitor crashes or loses power, the system should default to a safe shutdown, not continue uncontrolled operation). Another relevant concept is mode control and restriction – robots often have modes where higher speeds are allowed only when no human is nearby, enforced by external sensors. Translating this to AI, an extrinsic safeguard might enforce mode restrictions (e.g., an autonomous vehicle can only enter autonomous mode in certain conditions or areas, and the monitor will refuse or exit autonomy if conditions are unsafe). Industrial automation also teaches us about testing and validation of safety functions: these are often exhaustively tested under all foreseeable fault conditions. For an AI monitor, that implies extensive scenario testing (both simulations and real-

world trials) where the AI is forced into potential failure behaviours to ensure the monitor catches them. One gap in industrial domain is dealing with AI's adaptability – if the AI learns or updates behaviour on the fly (online learning), the monitor's fixed safety rules must still remain valid and not be undermined. Ensuring the monitor's ruleset covers a moving target of AI behaviour could be an added requirement in such systems. Overall, the industrial perspective contributes requirements for

(a) fail-safe design of the safeguard (default to safe on failure),

(b) extremely high reliability and quick reaction to detected hazards,

(c) independent and redundant monitoring channels (both in logic and hardware), and

(d) thorough validation of the safety system across all normal and fault scenarios.

D. Synthesis

Across these domains, common themes emerge. A runtime safeguard must be independent from the primary AI controller (both to avoid interference and to survive faults that affect the AI), it must have the authority and capability to enforce safety actions, and it must be engineered to a higher level of assurance since it carries the burden of preventing catastrophic failure. It should also be comprehensive in its monitoring – covering all critical variables that could lead to an accident – yet as simple as possible in design to allow rigorous certification. We also see the importance of a well-defined fallback strategy: simply detecting a problem is not enough; the system needs a clear course of action to transition to safe mode. In the next section, we distil these insights into a set of proposed requirements for extrinsic safeguarding mechanisms, with an eye toward meeting certification criteria such as DO-178C DAL A and equivalent standards.

V. PROPOSED REQUIREMENTS FOR CERTIFIABLE SAFEGUARDS

Drawing on the cross-domain insights, we propose a structured set of requirements for an extrinsic runtime assurance mechanism (safety monitor and fallback controller) to be certifiable for use in safety-critical Physical AI systems. Each requirement is framed to facilitate compliance with standards like DO-178C (DAL A), IEC 61508 (SIL3/SIL4), or ISO 26262 (ASIL D), meaning the requirements emphasise determinism, verifiability, and completeness.

1) Monitor-Controller Independence:

The safety monitor (and any backup controller it triggers) shall be functionally and structurally independent from the primary AI controller.

This means it should run on isolated computing resources or partitions, use separate logic, and not rely on the primary controller's correct operation for its own functioning. Rationale: Independence ensures that faults or adverse conditions affecting the AI (e.g., software bugs, electromagnetic interference, or adversarial inputs) do not simultaneously disable the safeguard. For example, if the AI software hangs, the monitor should continue running unhindered on a separate processor or real-

time task. Independence extends to inputs – where possible, the monitor should cross-check critical sensor data through diverse sources or validated input paths. Any shared inputs should be sanitised (e.g., via an Input Manager that provides assured, validated data to the monitor). This requirement aligns with the design assurance principle that the safety function has minimal common cause vulnerabilities with the function it protects.

2) Safety Constraint Definition and Runtime Enforceability

The system integrator (or designer) shall define a set of explicit safety constraints or invariants that the monitor will enforce at runtime.

These constraints must cover all identified hazardous conditions (e.g., “vehicle must not exceed $X \text{ m/s}^2$ deceleration”, “robot arm must not exert force above $Y \text{ N}$ when humans are detected”, “drone must remain within geofence boundaries”). The monitor shall continuously evaluate these constraints against the system's state and the AI's commanded actions. Critically, each constraint must be runtime-observable and enforceable, for instance, a constraint on a physical parameter that the monitor can measure or compute in real time, and if a violation is impending or occurs, the monitor can take a definitive action to enforce safety (such as clipping the command, activating an override, or engaging a shutdown). This requirement ensures that abstract safety goals (from hazard analysis) are instantiated into concrete real-time checks. It also implies that the monitor's design includes the necessary sensing or estimation capabilities to know when a constraint is being violated.

For example, enforcing a “no collision” constraint could involve monitors on time-to-collision using sensor inputs. The runtime enforceability aspect means we avoid any safety requirement that cannot be acted upon until after the fact – the monitor's interventions are happening in the control loop, not just producing warnings. In certifiable terms, these constraints become high-level requirements for the monitor subsystem, traceable to top-level safety objectives.

3) Hazard Coverage

The set of monitored safety constraints shall be sufficiently comprehensive to cover all foreseeable hazardous scenarios of the system.

In other words, for each identified hazard or unsafe failure mode in the system's safety analysis, there should be at least one monitor condition or rule that would detect and mitigate that hazard. If the AI can fail in ways that are not directly observable by a single parameter, the architecture should include multiple monitors or a multi-faceted condition. For example, an autonomous vehicle's hazards might include collision, rollover, or going off-road; a comprehensive monitor set might include forward collision monitoring, lateral stability monitoring, and geo-fence/road-edge monitoring. The ASTM F3269-21 guidance uses the term “RTA system coverage” to indicate the monitor should fully implement the desired safety coverage. This requirement implies a rigorous mapping from hazard analysis (PHA, HAZOP, FTA, etc.) to monitor design. Any hazard not covered by a monitor is essentially unmitigated at runtime and would undermine the safety case. Note that “foreseeable” is important; we cannot guarantee absolute completeness for unknown unknowns, but a margin of

conservatism is recommended (monitors should be designed with some tolerance to catch anomalies outside the precise scenarios considered). This requirement is tied to certification expectations that one must either eliminate or control every identified hazard. The runtime safeguard is the control for hazards that the AI's nominal logic cannot guarantee to avoid.

4) Proven Correctness and Simplicity of the Monitor

The safety monitor (and fallback controller) shall be kept as simple as possible and developed with formally verifiable or highly testable logic, to enable a strong assurance of correctness.

Because the monitor is a critical component (the safety of the entire system depends on it initiating the correct action in emergencies [13]), it must have a correctness assurance commensurate with its criticality (DAL A / SIL 4 level). In practice, this means the monitor's algorithms (e.g. threshold checks, rule-based logic, or simple control laws for fallback) should be amenable to complete requirements-based testing (100% path and coverage testing) and even formal verification techniques. For instance, if the monitor uses a rule like "IF distance < d_min THEN brake," one can exhaustively analyse that rule for all conditions. Recent work in runtime verification shows monitors can be specified with formal logic constraints which are machine-checked for consistency[5]. We recommend using similar approaches: e.g., use formal specification languages or model checking on the monitor's design to prove properties like "the monitor will always trigger at or before the constraint violation" and "the monitor will not trigger spuriously outside of designed conditions" (or at least with bounded false-alarm rate). This requirement aligns with DO-178C objectives for high-criticality software, e.g., traceability from requirements to code, and verification that the code conforms exactly to requirements. The simpler and more deterministic the monitor, the easier it is to provide that evidence. By contrast, the primary AI might be complex or non-deterministic (and possibly DAL B/C or uncertified), but the monitor must be airtight in its correctness to justify accepting the AI's presence.

5) Monitor Response Timeliness

The runtime assurance mechanism shall have a bounded response time to any safety constraint violation, guaranteed to be shorter than the time to collision (or constraint violation).

This means from the moment a hazard condition is detected (or predicted) by the monitor to the moment the system is brought to a safe state, the elapsed time is within the tolerances defined by the hazard analysis. For example, if a robot exceeding speed limit will cause injury within 100ms of contact, the monitor might be required to intervene within, say, 50ms of detecting overspeed. This requirement entails performing worst-case execution time (WCET) analysis on the monitor software, ensuring real-time scheduling guarantees, and often requiring high priority for the monitor's tasks. It might also constrain the complexity of what the monitor computes (favouring simple, fast checks). From a certification standpoint, demonstrating timing determinism and meeting deadlines is crucial (e.g., DO-178C and DO-254 would require verifying that the hardware/software can react in time under all conditions).

Monitor response time also relates to actuator dynamics, for instance, if using a backup braking system, that system must

actuate quickly enough. In summary, the monitor's latency from hazard onset to mitigation shall be analysed and verified to always prevent the prohibited condition from fully materialising.

6) Safe Fallback and Control Authority

Upon detecting a violation or an impending unsafe state, the mechanism shall trigger a pre-defined safe fallback action and assume control authority to execute that action.

The fallback action could range from emergency stop to a graceful degradation mode, but it must be effective in reducing the risk. Importantly, the system design must ensure the monitor can take control when needed, for example, a safety override that cuts off the AI's commands and routes control to a simplified backup controller, or direct control over actuators (like a parallel brake actuator in a car, or an independent kill switch on a motor). This requirement addresses two aspects:

(a) Fallback strategy completeness: The chosen fallback must cover all hazardous outcomes. If simply stopping covers all, fine; if some hazards require an alternative (like switching to manual remote control or parachute deployment for a drone), those mechanisms must be in place. The fallback should be capable of handling the situation after the AI is disengaged – e.g., bringing a vehicle to a stop safely, not abruptly in a way that causes other dangers unless absolutely necessary. According to NASA's RTA recommendations, the fallback (termed "Backup function") should cover all foreseeable operating conditions of the subject system [13].

(b) Authority: There can be no ambiguity that the safety mechanism's commands have priority over the AI's. This might be implemented via hardware interlocks or software priority: for instance, a monitor command could directly modulate throttle/braking, bypassing the AI's output. In a robotic system, the safety PLC can cut power to drives independent of the robot's controller. This priority ensures that once a hazard is detected, the primary controller cannot "fight" the corrective action. For certification, demonstrating this authority often means showing a separate control path in schematics and that the actuator will obey the safety input even if the main controller disagrees.

7) Fault Tolerance and Fail-Safe Design

The safeguard system itself shall be designed with appropriate fault tolerance to random hardware failures and shall fail safe if it fails.

Given that the monitor/fallback is now a critical subsystem, its failure should be considered in the safety analysis. The requirement could be that the monitor meets a certain reliability (e.g., $\geq 10^{-9}$ failure/hour for DAL A, or SIL3/4 levels of hardware reliability). Techniques include redundancy within the monitor (dual-channel comparison, self-checks) or at minimum, designing the monitor to default to a safe output on failure. For example, if a monitor process crashes, a watchdog timer could trigger the fallback anyway (assuming worst case that something is wrong).

Many industrial safety systems implement two-channel architecture where any discrepancy triggers a stop, an approach that could be mirrored in a high-assurance monitor. Also, the

communication between monitor and actuators should be safety-rated (e.g., use of a safe bus or protocol with heartbeat; loss of heartbeat triggers safe shutdown). In essence, this requirement is about making the monitor as robust as any primary safety system would be in traditional engineering.

Certification evidence would include reliability analyses (FMEDA, etc.) of the monitor and demonstration that any single failure in the monitor does not create an unsafe situation (ideally it leads to a controlled stop). This might also involve proving absence of “deadly juggle” situations, for instance, the monitor should not get stuck in an indefinite loop of handing back control to a faulty AI and re-intervening; once it decides to intervene, it should keep control until a clearly safe condition is restored and perhaps only return control under defined conditions (to avoid oscillation).

8) Minimal Interference

The safeguard mechanism should be designed to intervene with (or degrade) the primary system’s performance only when necessary for safety.

Under normal (safe) operating conditions, the monitor stays passive or in observation mode, and the AI can control the system freely within the safe envelope. This requirement is more about system usability; it ensures the safeguard does not needlessly constrain the AI, which could reduce the benefits of using AI in the first place. One way to implement this is to set thresholds such that normal variance doesn’t trigger the fallback; the threshold could have a buffer (for example, intervene at 90% of a truly dangerous value, but not at 50%). However, this must be balanced with not missing real hazards. The principle is that the monitor’s logic should be specific to safety violations to avoid false positives that could cause unnecessary interruptions. In certification terms, demonstrating this might involve testing that the monitor does not trigger during a large set of nominal scenarios (showing false alarm rate is acceptably low), and that it only triggers for the scenarios where it is supposed to.

This requirement connects to stakeholder acceptance as well: if a safety mechanism intervenes too often or erroneously (so-called “nuisance trips”), operators might be tempted to disable it, which is an undesirable outcome. So, a certifiable safeguard must also be well-calibrated; a point also noted in NASA’s guidance, which allows that a higher safety criticality might tolerate more false positives [13], but the goal is an optimal trade-off.

9) Lifecycle Monitoring and Adaptation (Optional)

For AI systems that learn or whose environment evolves, the requirements should include a plan for updating or adapting the monitor’s rules over time without compromising safety.

While not strictly a design-time requirement, it is an operational requirement that touches certification if updates are needed (any changes to a DAL A monitor would require re-certification or at least a safety assessment). This can be addressed by designing the monitor with tuneable parameters that can be adjusted within safe bounds (with, say, offline analysis) or by deploying analytics that flag when the AI is operating in regimes not covered by the monitor’s assumptions. Essentially, there should be a requirement that the validity of the

monitor’s safety constraints is maintained throughout the system’s life – if new hazards are discovered in operation (per SOTIF, new unknowns become known), the system owner must update the safeguard accordingly. Certification authorities will expect a plan for configuration control of the safety system and perhaps re-validation after significant updates. This requirement ensures that the extrinsic safeguard keeps up with the AI’s evolution and the real world, maintaining trust over time.

B. Summary

Each of the above requirements is aimed at enabling the runtime safeguard to be treated as a certifiable safety component (e.g., one could create a DO-178C certification package or an ISO 26262 work product for the monitor subsystem satisfying these points). By fulfilling monitor-controller independence and high assurance development, we ensure the monitor can be trusted to DAL A standards. By having clearly defined safety constraints and fallback actions, we make the system’s safety behaviour predictable and verifiable. The combination of comprehensive hazard coverage, rapid response, and fail-safe design makes the safeguard a robust mitigant in the overall safety case.

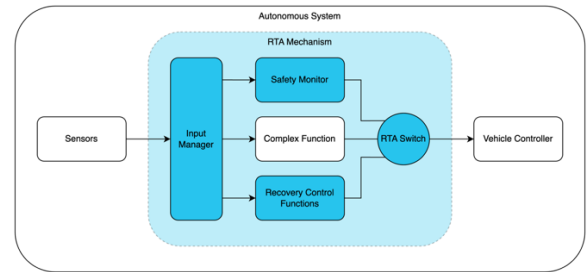


Fig. 2. Safeguarding mechanism architecture [4]

Figure 2 illustrates a reference architecture meeting these requirements, with an independent safety monitor watching the AI’s inputs and outputs, and a fallback controller that takes over through a safety switch when triggered.

VI. DISCUSSION

The introduction of extrinsic safeguarding mechanisms raises several important discussions regarding system design, certification, and stakeholder acceptance. We discuss some implications and potential challenges in implementing the proposed requirements across different domains.

Completeness vs. Complexity Trade-off: One practical challenge is ensuring that the monitor’s set of safety constraints is complete (covers all hazards) without becoming so complex that it defeats the purpose. If one attempted to monitor “every aspect” of a complex AI, the monitor might itself start to approach the complexity of the AI, which is undesirable. The art of RTA design is to choose a minimal sufficient set of constraints that provide a robust safety net. This often involves engineering judgment and iterative hazard analysis. For example, instead of monitoring every internal variable of an AI controller, one might monitor just the outcomes (like vehicle acceleration, distances to obstacles, path deviations) which are more tractable. Our requirements assume that such a set can be defined, but in practice designers must be wary of requirement

creep. One mitigation is to leverage formal hazard analysis techniques to justify that certain scenarios are either non-credible or covered by others, so the monitor remains focused. Additionally, any constraint added to the monitor should be traceable to a hazard, avoiding arbitrary rules that unnecessarily restrict the AI.

Integration and Performance Impact: Implementing an extrinsic monitor requires integration at both software and hardware levels. For instance, adding a parallel brake controller in a car or an independent flight termination system in a drone might introduce extra components and cost. There could be performance impacts too, e.g., slight delays due to monitor checks, or conservative limits that sometimes reduce efficiency. However, these are generally small trade-offs for safety. A potential issue is if the AI and monitor disagree on control in edge cases – could there be oscillations or instability? This is why we require that the monitor’s authority is absolute once engaged (to avoid fighting controllers). Designers often implement a hysteresis or hold-down: once the backup takes over, the AI can only regain control after a clear reset condition. Still, transitions must be smooth where possible (e.g., handing control to a safe controller rather than simply cutting off all control abruptly, except in true emergency stops). In testing, significant effort is needed to validate that these transitions do not themselves cause hazards (for example, an emergency stop on a highway is safe for avoiding a crash but introduces a new risk of being rear-ended; hence system-level analysis must weigh such consequences).

Certification Pathways: The proposed requirements align with certification expectations, but regulatory frameworks are still catching up on AI. One discussion point is how certification authorities like the FAA, FDA, or automotive regulators will view an AI+Monitor system. Ideally, if the monitor is shown to meet DAL A (or equivalent) and completely mitigate certain failure classes of the AI, authorities could allow the AI to be treated with a lower rigor. ASTM F3269-21 and similar efforts suggest an emerging pathway: partition the system into a high-assurance part (monitor) and a complex part (AI function) and provide a safety case that the high-assurance part keeps the overall system behaviour safe [5]. The question is whether this is sufficient in practice. There may be certification credit given for the monitor (i.e., you don’t have to certify the AI to the same level if the monitor is proven). However, this depends on trust in the monitor’s completeness. If any credible scenario exists where the AI could cause an accident that the monitor doesn’t prevent, the certification authority might still require fixes in the AI. Therefore, demonstrating requirement #3 (Complete Hazard Coverage) to authorities is vital, often via extensive testing, simulation, and analysis evidence. We expect certification guidance to evolve, possibly with standards specifically targeting runtime assurance systems (beyond ASTM’s practice, maybe in future DO-xx guidance or ISO TSs). Our requirements could serve as a basis for such future standards.

Human Factors and Trust: An interesting aspect is how extrinsic safeguards influence human stakeholder trust. On one hand, knowing that an independent safety system is always watching the AI can greatly enhance trust, similar to how drivers trust airbags or ABS to step in during emergencies. On the other hand, complexity of the overall system increases, and

explaining the combined behaviour to users or operators can be non-trivial. It will be important to communicate the presence and role of the runtime monitor to stakeholders. For example, an autonomous car might inform passengers that “the vehicle has an independent safety system that will ensure a safe stop if anything goes wrong.” In industrial settings, operators are trained on e-stops and safety circuits; similarly, they should understand the AI’s safeguard (perhaps how to manually engage it or the conditions under which it triggers). From a trust framework perspective, extrinsic safeguards directly address the Technical Robustness and Safety pillar of trustworthy AI [3] by providing resilience (ability to handle faults) and a fallback plan. In some cases, the existence of a monitor can also simplify accountability – since the monitor logs interventions, one can analyse incidents to see whether the AI misbehaved and how the safeguard responded, aiding transparency.

Limitations and Future Work: We acknowledge that runtime assurance is not a panacea. There are limits to what an external monitor can do, especially if the AI operates in very high-dimensional ways. For instance, if an AI makes a subtle error that doesn’t violate any easily observable safety constraint until it’s too late, the monitor might not catch it. An example could be a strategic planning error by an AI (choosing an unsafe route far in advance), if no immediate constraint is broken until a hazard is unavoidable, the monitor might only trigger when avoidance is no longer possible. To address this, research is ongoing into more predictive monitors that can forecast future states (e.g., using reachability analysis or worst-case simulations on a simpler model) to give earlier warnings. This raises another challenge: ensuring the monitor itself doesn’t become too predictive and complex (which could undermine its certifiability). Striking that balance is a frontier for RTA research. Another limitation is that extrinsic safeguards generally assume a known set of safety goals; if an AI system has emergent behaviour that introduces a new type of hazard entirely outside the design basis, the monitor might not have a rule for it. Thus, combining runtime monitoring with continuous operational feedback (as in SOTIF’s iterative approach) is recommended, basically, use the monitor to handle what it knows, and use incident data to expand what it knows over time.

Future work will likely explore adaptive or learning-based monitors that can update themselves (with caution, since that complicates certification) and the use of formal methods to automatically generate monitors from safety specifications. Cross-domain collaboration is also important: techniques from aerospace (like formally verified monitors) could be ported to automotive; automotive’s mass-market perspective on cost and scalability could inform how to make these safeguards economical; robotics can contribute on safe physical human-robot interaction constraints, etc. We also foresee the development of standardised safety patterns or templates (similar to design patterns) for common runtime safeguards, e.g., a generic “speed governor” or “collision avoidance shield” that can be instantiated in different systems.

The case for extrinsic safeguards is strong, but implementing them requires careful systems engineering to satisfy stringent requirements without negating the benefits of AI. Our proposed requirements aim to facilitate that balance, by clearly delineating what a certifiable safeguard must achieve. The hope is that this

will accelerate the deployment of trustworthy physical AI – enabling society to reap the rewards of AI-driven systems with confidence that robust safety nets are in place.

VII. CONCLUSION

AI-enabled physical systems will increasingly become part of our transport, workplaces, and daily environments. Ensuring their safety and trustworthiness is paramount. This paper argued that extrinsic safeguarding mechanisms – independent runtime monitors and fallback controllers – are a crucial component in achieving system-level safety for autonomous vehicles, drones, robots and other “Physical AI” applications. We reviewed how existing standards and practices hint at the need for such mechanisms but do not fully spell out the requirements, especially in the context of modern AI that cannot be exhaustively verified pre-deployment. To fill this gap, we presented a cross-domain synthesis and proposed a comprehensive set of requirements for certifiable runtime assurance systems. These requirements address the independence, reliability, responsiveness, and effectiveness of safety monitors, and are intended to meet the highest levels of safety certification (DO-178C DAL A, ISO 61508 SIL4, etc.), thereby making them suitable for the most critical applications (automated flight, driverless cars, collaborative robots working with humans, and so on).

By enforcing monitor-controller independence, we ensure a fault in the AI does not cripple the safety net. By defining enforceable safety constraints and fallback strategies, we guarantee that when the AI’s behaviour would lead to danger, the system will detect it and intervene in time. We underscored the importance of designing the monitor to a high assurance standard – keeping it simple and verifiable – since it carries the burden of safety. Cross-domain insights, from aviation’s RTA architectures to automotive fail-safe manoeuvres and industrial safety circuits, all converged on the same principles encapsulated in our requirements. Furthermore, we discussed how these extrinsic safeguards support stakeholder trust frameworks by providing transparency and predictable safety measures (e.g., the notion of a “guardian” that is always watching the AI’s actions).

The implications of adopting extrinsic safeguards are significant. For industry, it means a path to certify and deploy AI functions that might otherwise be too risky – essentially enabling a trustworthy envelope around unproven AI. For regulators, it offers a concrete way to approve advanced autonomous systems by focusing on the runtime safety layer. For the public and end-users, it provides reassurance that AI is never in complete unchecked control: there is always an independent mechanism ready to prevent worst-case outcomes. Of course, this approach does not eliminate the need for good AI design; rather, it complements it, providing *defense in depth*.

In conclusion, extrinsic safeguarding mechanisms provide a viable and necessary path to Trustworthy Physical AI. They bridge the gap between the ambitious capabilities of AI and the uncompromising demands of safety engineering. By formalising requirements for these mechanisms, we enable a more systematic development and certification approach. The recommendations herein encourage industry stakeholders to adopt runtime assurance patterns and for standardisation bodies

to incorporate such requirements into future guidelines. If broadly applied, these mechanisms can significantly reduce the risks of autonomous systems, allowing society to benefit from AI innovations with confidence that robust safety nets are in place at all times. The ongoing evolution of standards like UL 4600 and ASTM F3269, and the attention of regulators to runtime safety, suggest that extrinsic safeguards will become an integral part of the design vocabulary for safe autonomous systems. Ultimately, trust in Physical AI will be earned not only by how intelligent these systems are, but by how reliably they can guard against their own failures, and extrinsic safeguard mechanisms are a key ingredient in providing that reliability.

REFERENCES

- [1] I. Habli *et al.*, “The BIG Argument for AI Safety Cases,” 2025.
- [2] J. McDermid, K. Clegg, Y. Jia, and I. Habli, “AI GUARDRAILS: CONCEPTS, MODELS AND METHODS,” York, Dec. 2024. Accessed: Jun. 09, 2025. [Online]. Available: <https://www.york.ac.uk/media/assuring-autonomy/news/AI%20Guardrails%20CfAA%20White%20Paper.pdf>
- [3] European Commission High-Level Expert Group on AI, “Ethics guidelines for trustworthy AI,” <https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai>. [Online]. Available: <https://ec.europa.eu/futurium/en/ethics-guidelines-trustworthy-ai/register-piloting-process-0>
- [4] ASTM, “Standard Practice for Methods to Safely Bound Behavior of Aircraft Systems Containing Complex Functions Using Run-Time Assurance,” 2021, doi: 10.1520/F3269-21.
- [5] P. Nagarajan, S. K. Kannan, M. E. Vukas, C. Torens, and G. F. Wilber, “ASTM F3269 - an industry standard on run time assurance for aircraft systems,” in *AIAA Scitech 2021 Forum*, American Institute of Aeronautics and Astronautics Inc, AIAA, 2021, pp. 1–12. doi: 10.2514/6.2021-0525.
- [6] UL, “UL 4600: Standard for Evaluation of Autonomous Products,” 2020, *Underwriters Laboratories*. [Online]. Available: https://www.shopulstandards.com/ProductDetail.aspx?productId=UL4600_3_S_20230317
- [7] P. Koopman and P. Laplante, “UL 4600: What to Include in an Autonomous Vehicle Safety Case,” *Computer (Long Beach Calif)*, vol. 56, no. 5, pp. 101–104, May 2023, doi: 10.1109/MC.2023.3236171.
- [8] ISO, *ISO 21448:2022 Road vehicles — Safety of the intended functionality*. 2022.
- [9] ISO, *ISO 26262-1:2018 Road vehicles — Functional safety*. 2018.
- [10] ISO, *ISO 10218-1:2025 Robotics — Safety requirements*. 2025.
- [11] ISO, *ISO 13849-1:2023 Safety of machinery — Safety-related parts of control systems*. 2023.
- [12] IEC, *IEC 61508-1:2010 Functional safety of electrical/electronic/programmable electronic safety-related systems*. 2010.
- [13] G. Brat and G. Pai, “Runtime Assurance of Aeronautical Products: Preliminary Recommendations,” 2023. [Online]. Available: <http://www.sti.nasa.gov>
- [14] A. Desai, S. Ghosh, S. A. Seshia, N. Shankar, and A. Tiwari, “SOTER: A Runtime Assurance Framework for Programming Safe Robotics Systems,” Apr. 2018, [Online]. Available: <http://arxiv.org/abs/1808.07921>
- [15] K. Hobbs, M. Mote, M. Abate, S. Coogan, and E. Feron, “Run Time Assurance for Safety-Critical Systems: An Introduction to Safety Filtering Approaches for Complex Control Systems,” Apr. 2021, doi: 10.1109/MCS.2023.3234380.
- [16] SC-205, “DO-178C - Software Considerations in Airborne Systems and Equipment Certification,” 2011, *RTCA*.
- [17] SAE International, *Guidelines for Conducting the Safety Assessment Process on Civil Aircraft, Systems, and Equipment ARP4761A*. 2023.

